



# Tuyển tập Bài tập: Danh sách Liên kết (Linked List)

**Tác giả:** Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

**Biên soạn:** Được hỗ trợ và soạn thảo bởi Agentic AI tool

**Bản quyền © 2026 Duc-Minh Vu.** Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.



## Bảng Tổng hợp Bài tập Tuyển chọn

| STT | Bài toán                                    | Nền tảng     | Độ khó       | Kỹ thuật trọng tâm                              |
|-----|---------------------------------------------|--------------|--------------|-------------------------------------------------|
| 1   | <a href="#">Reverse Linked List</a>         | LeetCode 206 | ★ Cơ bản     | 3 con trỏ đảo liên kết $O(N)$ $O(1)$ RAM        |
| 2   | <a href="#">Middle of the Linked List</a>   | LeetCode 876 | ★ Cơ bản     | Con trỏ Nhanh - Chậm (Fast & Slow Pointers)     |
| 3   | <a href="#">Linked List Cycle</a>           | LeetCode 141 | ★ Cơ bản     | Thuật toán Rùa và Thỏ (Floyd's Cycle Detection) |
| 4   | <a href="#">Merge Two Sorted Lists</a>      | LeetCode 21  | ★ Cơ bản     | Ghép danh sách tăng dần & Kỹ thuật Dummy Head   |
| 5   | <a href="#">Remove Linked List Elements</a> | LeetCode 203 | ★ Cơ bản     | Xóa phần tử bằng Sentinel/Dummy Node            |
| 6   | <a href="#">Palindrome Linked List</a>      | LeetCode 234 | ★ Cơ bản     | Tìm trung điểm + Đảo ngược nửa sau danh sách    |
| 7   | <a href="#">Linked List Cycle II</a>        | LeetCode 142 | ★★ Trung cấp | Tìm điểm bắt đầu chu trình (Floyd's Algorithm)  |
| 8   | <a href="#">Reverse Linked List II</a>      | LeetCode 92  | ★★ Trung cấp | Đảo ngược một đoạn con $[L, R]$ in-place        |
| 9   | <a href="#">Remove Nth Node From End</a>    | LeetCode 19  | ★★ Trung cấp | Cửa sổ trượt con trỏ tạo khoảng cách $N$        |
| 10  | <a href="#">Reorder List</a>                | LeetCode 143 | ★★ Trung cấp | Fast-Slow + Reverse List + Interleaving Merge   |
| 11  | <a href="#">LRU Cache</a>                   | LeetCode 146 | ★★ Trung cấp | Hash Map + Doubly Linked List $O(1)$            |

| STT | Bài toán                                 | Nền tảng     | Độ khó         | Kỹ thuật trọng tâm                                           |
|-----|------------------------------------------|--------------|----------------|--------------------------------------------------------------|
| 12  | <a href="#">Sort List</a>                | LeetCode 148 | ★ ★ Trung cấp  | Merge Sort chia để trị trên Danh sách liên kết $O(N \log N)$ |
| 13  | <a href="#">Reverse Nodes in k-Group</a> | LeetCode 25  | ★ ★ ★ Nâng cao | Đảo ngược khối $K$ phần tử đệ quy / con trỏ kép              |
| 14  | <a href="#">Merge k Sorted Lists</a>     | LeetCode 23  | ★ ★ ★ Nâng cao | Hàng đợi ưu tiên Min-Heap $O(N \log K)$                      |
| 15  | <a href="#">LFU Cache</a>                | LeetCode 460 | ★ ★ ★ Nâng cao | Tần suất kép (Frequency Map + Doubly Linked List)            |
| 16  | <a href="#">Josephus Queries</a>         | CSES 2164    | ★ ★ ★ Nâng cao | Toán học đệ quy mô phỏng vòng tròn liên kết                  |



## HƯỚNG DẪN GIẢI CHI TIẾT & GỢI Ý CHIẾN THUẬT

### 1. Nhóm Bài tập Cơ bản (6 bài)

#### Bài 1: [Reverse Linked List](#)

- **Đề bài tóm tắt:** Cho con trỏ đầu danh sách liên kết đơn `head`, hãy đảo ngược danh sách và trả về con trỏ đầu mới.
- **Phân tích & Thuật toán:**
- Dùng 3 con trỏ: `prev = nullptr`, `curr = head`, và `nxt = nullptr`.
- Trong vòng lặp: lưu `nxt = curr->next`, bẻ ngược `curr->next = prev`, tịnh tiến `prev = curr`, rồi `curr = nxt`.
- Kết thúc khi `curr == nullptr`, trả về `prev`.
- Độ phức tạp:  $O(N)$  thời gian,  $O(1)$  bộ nhớ phụ trợ.

#### Bài 2: [Middle of the Linked List](#)

- **Đề bài tóm tắt:** Cho `head`, tìm nút nằm chính giữa danh sách. Nếu có 2 nút giữa, trả về nút thứ 2.
- **Phân tích & Thuật toán:**
- Khởi tạo 2 con trỏ cùng tại `head`: `slow = head`, `fast = head`.
- Mỗi bước: `slow` đi 1 bước (`slow = slow->next`), `fast` đi 2 bước (`fast = fast->next->next`).
- Vòng lặp dừng khi `fast == nullptr` hoặc `fast->next == nullptr`. Lúc này `slow` đứng chính xác tại trung điểm cần tìm.
- Độ phức tạp:  $O(N)$  thời gian, duyệt đúng 1 lượt.

### Bài 3: [Linked List Cycle](#)

- **Đề bài tóm tắt:** Xác định xem danh sách liên kết có tồn tại chu trình (cycle) hay không mà không được sửa đổi cấu trúc node.
  - **Phân tích & Thuật toán:**
  - Dùng thuật toán Rùa và Thỏ (Floyd's Cycle-Finding Algorithm).
  - Nếu danh sách tuyến tính, `fast` sẽ chạm `nullptr`.
  - Nếu danh sách có vòng lặp, vì `fast` di chuyển nhanh hơn `slow` 1 node mỗi bước, khoảng cách giữa chúng trong vòng tròn sẽ giảm dần từng đơn vị một cho đến khi `slow == fast`.
  - Độ phức tạp:  $O(N)$  thời gian,  $O(1)$  bộ nhớ.
- 

### Bài 4: [Merge Two Sorted Lists](#)

- **Đề bài tóm tắt:** Hợp nhất hai danh sách liên kết đã sắp xếp tăng dần thành một danh sách duy nhất cũng có thứ tự tăng dần.
  - **Phân tích & Thuật toán:**
  - Sử dụng một nút giả `ListNode dummy(0);` và con trỏ `tail = &dummy;`.
  - So sánh `l1->val` và `l2->val`, gán `tail->next` vào node nhỏ hơn, sau đó tịnh tiến con trỏ tương ứng.
  - Sau vòng lặp, nếu một trong hai danh sách còn phần tử, nối trực tiếp `tail->next = (l1 ? l1 : l2);`.
  - Trả về `dummy.next`.
- 

### Bài 5: [Remove Linked List Elements](#)

- **Đề bài tóm tắt:** Xóa tất cả các nút trong danh sách có giá trị bằng `val`.
  - **Phân tích & Thuật toán:**
  - Khó khăn lớn nhất là nút cần xóa có thể nằm ngay tại `head`.
  - Khởi tạo `ListNode dummy(0, head);` và `prev = &dummy;`.
  - Duyệt `while (prev->next)`: nếu `prev->next->val == val`, bỏ qua node đó bằng `prev->next = prev->next->next;` (có thể giải phóng bộ nhớ nếu cần); ngược lại di chuyển `prev = prev->next;`.
- 

### Bài 6: [Palindrome Linked List](#)

- **Đề bài tóm tắt:** Kiểm tra xem danh sách liên kết đơn có đối xứng (palindrome) hay không với thời gian  $O(N)$  và bộ nhớ  $O(1)$ .
  - **Phân tích & Thuật toán:**
  - Bước 1: Dùng con trỏ nhanh chậm tìm điểm giữa danh sách.
  - Bước 2: Đảo ngược nửa sau của danh sách liên kết (từ `slow` trở đi).
  - Bước 3: So sánh từng cặp giá trị từ đầu (`head`) và từ đuôi đã đảo (`secondHalf`).
  - Bước 4 (Tùy chọn): Đảo ngược lại nửa sau để khôi phục danh sách ban đầu.
-

## 2. Nhóm Bài tập Trung cấp (6 bài)

---

### Bài 7: [Linked List Cycle II](#)

- **Đề bài tóm tắt:** Tìm và trả về chính xác nút bắt đầu chu trình, nếu không có chu trình trả về `nullptr`.
  - **Phân tích & Thuật toán:**
  - Dùng thuật toán Floyd để tìm điểm gặp nhau  $M$  của `slow` và `fast`.
  - Đặt một con trỏ `ptr1 = head`, giữ nguyên `ptr2 = M`.
  - Cả hai con trỏ cùng bước 1 bước mỗi lần. Điểm gặp nhau tiếp theo chính là Nút bắt đầu chu trình (theo chứng minh  $F = (k - 1)C + (C - a)$  trong phần lý thuyết).
- 

### Bài 8: [Reverse Linked List II](#)

- **Đề bài tóm tắt:** Cho `head` và hai số nguyên  $L, R$  ( $1 \leq L \leq R \leq N$ ), đảo ngược đoạn danh sách từ vị trí  $L$  đến  $R$ .
  - **Phân tích & Thuật toán:**
  - Dùng Dummy Node `dummy(0, head)`.
  - Cho con trỏ `prev` nhảy  $L - 1$  bước để đứng ngay trước vị trí bắt đầu đảo.
  - Node `curr = prev->next` là nút đầu tiên của đoạn đảo.
  - Lặp  $R - L$  lần kỹ thuật rút nút tiếp theo chèn lên đầu đoạn:  

```
cpp ListNode* nxt = curr->next; curr->next = nxt->next; nxt->next = prev->next; prev->next =  
nxt;
```
  - Độ phức tạp:  $O(N)$  thời gian, duyệt đúng 1 lượt in-place.
- 

### Bài 9: [Remove Nth Node From End of List](#)

- **Đề bài tóm tắt:** Xóa nút thứ  $N$  tính từ cuối danh sách liên kết trở về trước trong đúng 1 lượt duyệt.
  - **Phân tích & Thuật toán:**
  - Sử dụng Dummy Node `dummy(0, head)`.
  - Đặt hai con trỏ `first = &dummy` và `second = &dummy`.
  - Cho `first` nhảy trước  $N + 1$  bước.
  - Sau đó cho cả `first` và `second` cùng nhảy từng bước một cho đến khi `first == nullptr`.
  - Lúc này, `second` đang đứng ngay trước nút cần xóa: `second->next = second->next->next;`.
- 

### Bài 10: [Reorder List](#)

- **Đề bài tóm tắt:** Biến đổi danh sách  $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$  thành  $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow \dots$ .
- **Phân tích & Thuật toán:**
- Bước 1: Tìm trung điểm danh sách bằng Fast & Slow pointer. Cắt đôi danh sách tại trung điểm bằng cách gán `mid->next = nullptr`.
- Bước 2: Đảo ngược nửa thứ hai của danh sách.

- Bước 3: Ghép xen kẽ (Interleave) từng nút của hai nửa lại với nhau.

---

## Bài 11: [LRU Cache](#)

- **Đề bài tóm tắt:** Thiết kế cấu trúc dữ liệu Bộ nhớ đệm LRU hỗ trợ `get(key)` và `put(key, value)` trong thời gian  $O(1)$ .
- **Phân tích & Thuật toán:**
- Kết hợp bảng băm `unordered_map<int, DNode*>` và Danh sách liên kết đôi có hai nút lính canh `head` và `tail`.
- Mọi thao tác truy cập đưa nút lên vị trí kế sau `head` (MRU).
- Khi kích thước vượt `capacity`, loại bỏ nút đứng trước `tail` (LRU) và xóa khỏi map.

---

## Bài 12: [Sort List](#)

- **Đề bài tóm tắt:** Sắp xếp danh sách liên kết trong thời gian  $O(N \log N)$  và bộ nhớ phụ trợ  $O(1)$  hoặc  $O(\log N)$ .
- **Phân tích & Thuật toán:**
- Áp dụng thuật toán **Merge Sort trên Danh sách Liên kết**:
  1. Dùng con trỏ nhanh chậm tìm trung điểm.
  2. Cắt danh sách thành 2 nửa độc lập.
  3. đệ quy sắp xếp từng nửa `sortList(left)` và `sortList(right)`.
  4. Dùng hàm `mergeTwoLists` để trộn 2 nửa đã sắp xếp lại với nhau.

---

## 3. Nhóm Bài tập Nâng cao (4 bài)

---

### Bài 13: [Reverse Nodes in k-Group](#)

- **Đề bài tóm tắt:** Đảo ngược danh sách liên kết theo từng nhóm  $K$  phần tử một. Nếu nhóm cuối không đủ  $K$  phần tử thì giữ nguyên thứ tự ban đầu.
- **Phân tích & Thuật toán:**
- Kiểm tra xem đoạn hiện tại có đủ  $K$  node không. Nếu không đủ, trả về ngay `head`.
- Nếu đủ  $K$  node, dùng kỹ thuật 3 con trỏ đảo ngược đúng  $K$  phần tử đầu tiên.
- Nút đầu ban đầu (`head`) sau khi đảo trở thành đuôi của nhóm, gán `head->next = reverseKGroup(curr, k);` để nối với các nhóm kế tiếp.

---

### Bài 14: [Merge k Sorted Lists](#)

- **Đề bài tóm tắt:** Hợp nhất  $K$  danh sách liên kết đã sắp xếp thành một danh sách sắp xếp duy nhất.
- **Phân tích & Thuật toán:**
- Đưa con trỏ đầu tiên của mỗi danh sách vào một `std::priority_queue` (Min-Heap) theo giá trị `val`.

- Mỗi bước rút phần tử nhỏ nhất từ Heap nối vào danh sách kết quả, nếu nút vừa rút có `next`, đẩy `next` vào Heap.
- Tổng số phần tử là  $N$ . Mỗi phần tử được đưa vào và lấy ra khỏi Heap kích thước tối đa  $K$  đúng 1 lần  $\implies$  Độ phức tạp thời gian:  $O(N \log K)$ , bộ nhớ  $O(K)$ .

## Bài 15: [LFU Cache](#)

- **Đề bài tóm tắt:** Thiết kế bộ đệm LFU (Least Frequently Used) đào thải phần tử có tần suất truy cập ít nhất. Nếu trùng tần suất, đào thải phần tử ít được dùng gần nhất (LRU).
- **Phân tích & Thuật toán:**
- Cần 3 cấu trúc dữ liệu:
  1. `keyTable` : map `key -> (value, freq)`.
  2. `freqTable` : map `freq -> DoublyLinkedList`, lưu danh sách các key có cùng tần suất theo thứ tự thời gian.
  3. `minFreq` : biến lưu tần suất nhỏ nhất hiện tại trong cache.
- Cả `get` và `put` đều đạt độ phức tạp  $O(1)$ .

## Bài 16: [CSES 2164 - Josephus Queries](#)

- **Đề bài tóm tắt:** Cho  $N$  đứa trẻ xếp thành vòng tròn đánh số từ 1 đến  $N$ . Bắt đầu từ người số 2, lần lượt loại bỏ các người ở vị trí cách nhau 1 người. Cho  $Q$  truy vấn ( $Q \leq 10^5$ ), mỗi truy vấn hỏi người bị loại thứ  $K$  mang số hiệu bao nhiêu ( $N \leq 10^9$ ).
- **Phân tích & Thuật toán:**
- Với  $N \leq 10^9$ , mô phỏng bằng Danh sách liên kết vòng sẽ bị TLE và MLE ( $O(N)$ ).
- Phân tích quy luật chia để trị: Sau một vòng duyệt loại bỏ hết các số chẵn, số lượng người giảm còn  $\lfloor N/2 \rfloor$ .
- Chuyển đổi trạng thái đệ quy trong  $O(\log N)$  mỗi truy vấn:
  - Nếu  $K \leq \lfloor N/2 \rfloor$ , người bị loại thứ  $K$  là số  $2K$ .
  - Nếu  $K > \lfloor N/2 \rfloor$ , bài toán quy về trường hợp kích thước nhỏ hơn với độ lệch (offset) phụ thuộc vào tính chẵn lẻ của  $N$ .
- Tổng độ phức tạp:  $O(Q \log N)$  tối ưu tuyệt đối.



### Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.